

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

A.R.Yuvansri

192421029

3. Motion Detection and Tracking System

FULL QUESTION:

Problem Statement:

A surveillance system must:

- Read video input
- Detect motion between consecutive frames
- Track moving objects
- Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Requirement: Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Detailed Solution & Architectural Breakdown

1. Theoretical Approach & Methodology:

To effectively satisfy the requirements under dynamic background conditions (such as swaying trees, light variations, or shadows), simple frame differencing alone is insufficient as it generates excessive false positives. Instead, an adaptive Mixture of Gaussians (MOG2) background subtraction approach is utilized alongside morphological filtering and Kalman Filter / Hungarian algorithm tracking (Centroid Tracking).

- **Preprocessing & Noise Suppression:** Convert incoming frames to grayscale and apply Gaussian Blurring (5x5 kernel) to smooth high-frequency spatial noise.
- **Dynamic Background Modeling:** Maintain a probabilistic model of each pixel background using Gaussian Mixture Models. Update the background model dynamically with learning rate α .
- **Motion Segmentation:** Compute absolute foreground mask. Apply thresholding followed by Morphological Opening (erosion then dilation) to eliminate speckle noise, and Morphological Closing to fill internal holes in detected moving blobs.
- **Contour & Bounding Box Extraction:** Extract connected components from the binary mask to locate candidate object regions.
- **Multi-Object Tracking:** Calculate centroid coordinates for each detected bounding box. Predict object trajectory using object IDs and Euclidean distance association.
- **Visualization:** Draw bounding boxes, centroid movement paths, and tracking IDs onto the original RGB video frame in near real-time.

System Pseudocode

```
SYSTEM MotionDetectionAndTracking:
    INPUT: video_stream_source (Camera Stream or File)
    OUTPUT: Real-time visual stream displaying tracked moving objects with
            bounding boxes

    DECLARE bg_subtractor = Initialize MOG2_BackgroundSubtractor(history=500,
```

```

varThreshold=16, detectShadows=True)
    DECLARE tracker = Initialize CentroidTracker(max_disappeared_frames=10,
max_distance=50)
    DECLARE kernel = Create MorphologicalKernel(shape=RECT, size=(5,5))

    FUNCTION MainProcess():
        cap = OpenVideoCapture(video_stream_source)
        IF NOT cap.is_opened():
            DISPLAY "Error: Unable to access video input stream."
            RETURN
        END IF

        WHILE cap.is_reading():
            ret, frame = cap.read_next_frame()
            IF NOT ret OR frame IS NULL:
                BREAK // End of video stream
            END IF

            // Step 1: Preprocessing
            gray_frame = ConvertToGrayscale(frame)
            blurred_frame = ApplyGaussianBlur(gray_frame, kernel_size=(5, 5),
sigma=1.5)

            // Step 2: Dynamic Background Subtraction & Filtering
            fg_mask = bg_subtractor.apply(blurred_frame, learning_rate=0.005)
            clean_mask = ApplyMorphologyEx(fg_mask, operation=OPEN,
kernel=kernel)
            clean_mask = ApplyMorphologyEx(clean_mask, operation=CLOSE,
kernel=kernel)
            clean_mask = ApplyBinarizationThreshold(clean_mask,
threshold_value=200)

            // Step 3: Contour Detection (Region Extraction)
            contours = FindContours(clean_mask, mode=RETR_EXTERNAL,
method=CHAIN_APPROX_SIMPLE)
            detections = EMPTY_LIST()

            FOR EACH contour IN contours:
                IF CalculateContourArea(contour) > MIN_MOTION_AREA_THRESHOLD
(e.g., 500 px):
                    (x, y, w, h) = GetBoundingRectangle(contour)
                    detections.append([x, y, x + w, y + h])
                END IF
            END FOR

            // Step 4: Object Tracking Logic
            tracked_objects = tracker.update(detections)

            // Step 5: Visualization and Display
            FOR EACH object IN tracked_objects:
                object_id = object.ID
                (centroid_x, centroid_y) = object.centroid
                (x1, y1, x2, y2) = object.bounding_box

                DrawRectangle(frame, pt1=(x1, y1), pt2=(x2, y2), color=BLACK,
thickness=2)
                DrawCircle(frame, center=(centroid_x, centroid_y), radius=4,
color=BLACK, fill=TRUE)
                DrawText(frame, text="ID: " + object_id, position=(x1, y1 - 10),
font_size=0.5)

```

```
END FOR

DisplayWindow("Real-Time Surveillance Motion Tracking", frame)

IF UserPressedKey('q'):
    BREAK // User termination request
END IF
END WHILE

cap.release()
CloseAllWindows()
END FUNCTION
END SYSTEM
```

6. Real-Time Object Detection Pipeline

FULL QUESTION:

Problem Statement:

A system must:

- Capture video frames
- Detect objects using a trained model
- Draw bounding boxes and labels
- Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Requirement: Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

Detailed Solution & Architectural Breakdown

1. Performance & Efficiency Strategy:

To ensure real-time performance (e.g., 30+ FPS) when running deep learning object detection models (such as YOLO or MobileNet-SSD), the software pipeline must decouple frame acquisition, model inference, and rendering. Single-threaded sequential loops often bottleneck on I/O operations.

- **Asynchronous Multi-Threading:** Use dedicated threads for video frame capture and inference to prevent frame buffer queuing delay.
- **Frame Skipping / Threaded Queuing:** Perform detection every N-th frame or drop stale frames from the queue to prioritize freshness over processing obsolete frames.
- **Pre-trained Model Batching & Quantization:** Utilize TensorRT/OpenVINO or FP16 quantized weights for rapid GPU/NPU execution.

- Non-Maximum Suppression (NMS): Filter out overlapping candidate bounding boxes based on Intersection over Union (IoU) thresholds to cleanly display distinct multiple objects.

System Pseudocode

```
MODULE RealTimeObjectDetectionPipeline:

    GLOBAL frame_queue = ThreadSafeQueue(maxsize=2)
    GLOBAL detection_results = ThreadSafeSharedObject()
    GLOBAL is_running = TRUE

    FUNCTION Thread1_FrameGrabber(camera_id):
        cap = OpenCameraStream(camera_id)
        WHILE is_running DO:
            ret, frame = cap.read()
            IF ret IS TRUE:
                IF frame_queue.is_full():
                    frame_queue.pop_oldest() // Prevent buffer latency
                END IF
                frame_queue.push(frame)
            END IF
        END WHILE
        cap.release()
    END FUNCTION

    FUNCTION Thread2_ObjectDetectionInference(model_path, confidence_threshold,
nms_threshold):
        model = LoadTrainedModel(model_path) // e.g., YOLOv8 / MobileNet-SSD

        WHILE is_running DO:
            IF NOT frame_queue.is_empty():
                current_frame = frame_queue.pop()

                // Preprocessing
                input_blob = PreprocessImage(current_frame, target_size=(416,
416), normalize=TRUE)

                // Forward Pass (Inference)
                raw_predictions = model.forward_pass(input_blob)

                // Post-processing (Filtering low confidence & NMS)
                boxes, scores, class_ids = ExtractDetections(raw_predictions,
confidence_threshold)
                final_boxes = ApplyNonMaximumSuppression(boxes, scores,
nms_threshold)

                // Update global thread-safe state
                detection_results.update(final_boxes, scores, class_ids)
            END IF
        END WHILE
    END FUNCTION

    FUNCTION MainDisplayThread():
        StartThread(Thread1_FrameGrabber, camera_id=0)
        StartThread(Thread2_ObjectDetectionInference,
model_path="yolo_weights.onnx", conf=0.5, nms=0.4)

        WHILE is_running DO:
```

```

        IF NOT frame_queue.is_empty():
            display_frame = frame_queue.get_latest()
            current_detections = detection_results.get_latest()

            // Visualization Logic
            FOR EACH box IN current_detections:
                (x, y, width, height) = box.coordinates
                label = box.class_name
                confidence = box.score

                // Render Bounding Box and Label
                DrawRectangle(display_frame, x, y, width, height,
color=BLACK, thickness=2)
                text_str = StringFormat("%s: %.2f%%", label, confidence *
100)
                DrawTextBackground(display_frame, text_str, x, y - 5,
font=TIMES_NEW_ROMAN)
            END FOR

            DisplayStreamWindow("Real-Time Object Detection", display_frame)

            IF CheckUserExitKey() IS TRUE:
                is_running = FALSE
            END IF
        END IF
    END WHILE

    StopAllThreads()
    DestroyAllWindows()
END FUNCTION
END MODULE

```

7. Image Difference Detection System

FULL QUESTION:

Problem Statement:

A system must:

- Read two images
- Compare them
- Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Requirement: Write pseudocode including preprocessing, comparison logic, and visualization.

Detailed Solution & Architectural Breakdown

1. Theoretical Approach & Methodology:

Comparing two images directly pixel-by-pixel causes false alerts due to slight lighting variations, camera sensor noise, or alignment mismatches. A robust structural comparison pipeline combines structural similarity metrics (SSIM) or absolute pixel-difference calculation with noise-suppression filtering.

- Preprocessing & Alignment Check: Verify dimensions match; resize or align using feature-matching (ORB/SIFT) if needed. Convert images to grayscale.
- Noise Suppression: Apply Bilateral Filtering or Gaussian Smoothing to eliminate high-frequency noise while preserving structural edges.
- Structural Difference / Frame Differencing: Compute Structural Similarity Index (SSIM) map or absolute pixel difference $|Image1 - Image2|$.
- Thresholding & Noise Clean-up: Apply Otsu's thresholding to generate a binary difference mask. Apply morphological opening to scrub isolated noise pixels.
- Visualization: Locate contours of difference regions and draw bounding boxes / transparent red overlays on the original input images.

System Pseudocode

```
SYSTEM ImageDifferenceDetector:
    INPUT: image_path_A, image_path_B
    OUTPUT: Visualization showing exact highlight regions of discrepancies

    FUNCTION MainDifferencePipeline(image_path_A, image_path_B):
        imgA = LoadImage(image_path_A)
        imgB = LoadImage(image_path_B)

        // Step 1: Validation and Dimension Normalization
        IF imgA.dimensions != imgB.dimensions THEN
            imgB = ResizeImage(imgB, target_width=imgA.width,
target_height=imgA.height)
        END IF

        // Step 2: Preprocessing and Noise Handling
        grayA = ConvertToGrayscale(imgA)
        grayB = ConvertToGrayscale(imgB)

        // Bilateral filter preserves sharp edges while smoothing sensor noise
        denoisedA = ApplyBilateralFilter(grayA, diameter=9, sigmaColor=75,
sigmaSpace=75)
        denoisedB = ApplyBilateralFilter(grayB, diameter=9, sigmaColor=75,
sigmaSpace=75)

        // Step 3: Difference Metric Computation (SSIM or Absolute Difference)
        (ssim_score, diff_map) = ComputeSSIM(denoisedA, denoisedB,
full_map=TRUE)
        // Convert floating point diff map [0, 1] to 8-bit image [0, 255]
        diff_image = ConvertTo8Bit(diff_map)

        // Step 4: Segmentation & Noise Clean-up
```

```

        binary_mask = ApplyOtsuThreshold(diff_image, invert=TRUE)
        kernel = MorphologicalKernel(shape=RECT, size=(3,3))
        clean_mask = ApplyMorphologyEx(binary_mask, operation=OPEN,
kernel=kernel) // Remove speckles

        // Step 5: Region Extraction and Visualization
        contours = FindContours(clean_mask, mode=RETR_EXTERNAL)
        highlight_canvas = CopyImage(imgB)

        FOR EACH contour IN contours DO:
            IF CalculateContourArea(contour) > MIN_DIFFERENCE_AREA_THRESHOLD
THEN
                (x, y, w, h) = GetBoundingBox(contour)
                DrawRectangle(highlight_canvas, (x, y), (x + w, y + h),
color=BLACK, thickness=2)

                // Draw semi-transparent overlay region
                ApplyColorOverlayRegion(highlight_canvas, contour, color=RED,
alpha=0.3)
            END IF
        END FOR

        DISPLAY "Structural Similarity Metric: " + ssim_score
        SaveImage("difference_output.png", highlight_canvas)
        DisplayImage("Difference Highlighting", highlight_canvas)
    END FUNCTION
END SYSTEM

```

8. Video Frame Extraction System

FULL QUESTION:

Problem Statement:

A system must:

- Read video input
- Extract frames at fixed intervals
- Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Requirement: Write structured pseudocode for frame selection and saving logic.

Detailed Solution & Architectural Breakdown

1. Storage Optimization & Redundancy Prevention Logic:

Extracting every single frame from a video results in massive storage overhead and severe redundancy (e.g., static scenes recorded over long durations). A smart frame extraction system

combines fixed temporal intervals with perceptual hashing (pHash) or SSIM similarity checks to drop identical/redundant frames, followed by efficient JPEG compression.

- Fixed Interval Sampling: Extract candidate frames based on time interval T (seconds) or frame step $N = \text{FPS} * T$.
- Perceptual Redundancy Check: Compute Perceptual Hash (pHash) or mean squared error (MSE) against the previously saved frame. If hamming distance $<$ threshold, discard as duplicate.
- Image Compression Optimization: Save frames as optimized JPEGs (quality factor 85-90) or WEBP format to minimize disk memory while preserving visual fidelity.
- Structured Directory Indexing: Output files with metadata-encoded filenames (e.g., frame_00123_timestamp_10.5s.jpg).

System Pseudocode

```
SYSTEM VideoFrameExtractionSystem:
    INPUT: input_video_path, extraction_interval_seconds, similarity_threshold,
    output_directory
    OUTPUT: Efficiently saved set of non-redundant image frames on disk

    FUNCTION ExtractAndSaveFrames():
        video = OpenVideo(input_video_path)
        IF NOT video.is_open():
            DISPLAY "Error: Cannot open video file."
            RETURN
        END IF

        fps = video.get_frames_per_second()
        frame_step = Round(fps * extraction_interval_seconds)

        frame_counter = 0
        saved_counter = 0
        last_saved_phash = NULL

        CREATE_DIRECTORY_IF_NOT_EXISTS(output_directory)

        WHILE video.has_next_frame():
            ret, current_frame = video.read_frame()
            IF NOT ret:
                BREAK
            END IF

            // Step 1: Interval Sampling Check
            IF (frame_counter MOD frame_step) == 0:

                // Step 2: Redundancy Prevention via Perceptual Hashing
                current_phash = ComputePerceptualHash(current_frame)

                IS_REDUNDANT = FALSE
                IF last_saved_phash IS NOT NULL THEN
                    hamming_dist = ComputeHammingDistance(current_phash,
last_saved_phash)
                    IF hamming_dist < similarity_threshold THEN
                        IS_REDUNDANT = TRUE
                    END IF
                END IF

            END IF

            frame_counter = frame_counter + 1
            IF NOT IS_REDUNDANT:
                saved_counter = saved_counter + 1
                SaveFrame(current_frame, output_directory, saved_counter)
            END IF
        END WHILE
```

```

        // Step 3: Efficient Storage Logic
        IF NOT IS_REDUNDANT THEN
            timestamp = frame_counter / fps
            filename = StringFormat("%s/frame_%06d_time_%.2fs.jpg",
output_directory, frame_counter, timestamp)

            // Compress and write to disk
            SaveImageWithCompression(filename, current_frame,
format="JPEG", quality=85)

            last_saved_phash = current_phash
            saved_counter = saved_counter + 1
        ELSE
            DISPLAY StringFormat("Skipped redundant frame at frame index
%d", frame_counter)
        END IF

    END IF

    frame_counter = frame_counter + 1
END WHILE

video.close()
DISPLAY StringFormat("Extraction complete. Processed %d frames, Saved %d
unique frames.", frame_counter, saved_counter)
END FUNCTION
END SYSTEM

```

14. Text Detection in Natural Scene Images

FULL QUESTION:

Problem Statement:

A system must:

- Read images
- Detect text regions
- Highlight detected areas

Constraints:

- Handle varying backgrounds
- Improve detection accuracy

Requirement: Write pseudocode including preprocessing, region detection, and visualization.

Detailed Solution & Architectural Breakdown

1. Theoretical Approach & Methodology:

Text detection in natural scene images presents severe challenges such as non-uniform illumination, complex textured backgrounds, arbitrary text orientations, and dynamic fonts.

Standard thresholding fails here. The optimal modern computer vision solution employs EAST (Efficient and Accurate Scene Text Detector) or MSER (Maximally Stable Extremal Regions) combined with Stroke Width Transform (SWT).

- **Preprocessing & Image Normalization:** Adjust contrast using CLAHE (Contrast Limited Adaptive Histogram Equalization) and apply bilateral filtering to preserve text edges while smoothing complex backgrounds.
- **Region Detection Strategy (Deep Learning Approach - EAST):** Input preprocessed image into a trained neural network designed to output pixel-wise text score maps and rotated bounding geometry (geometry map).
- **Non-Maximum Suppression (NMS):** Filter out overlapping bounding boxes and merge fragmented text candidates.
- **Visualization:** Render rotated/oriented polygon bounding boxes around identified scene text regions for accurate highlighting.

System Pseudocode

```
SYSTEM NaturalSceneTextDetector:
    INPUT: image_path, east_model_path, min_confidence_score
    OUTPUT: Visualized image highlighting detected text bounding regions

    FUNCTION DetectSceneText(image_path, east_model_path):
        image = LoadImage(image_path)
        (orig_h, orig_w) = image.dimensions

        // Step 1: Preprocessing for Varying Backgrounds & Lighting
        lab_image = ConvertToLAB(image)
        l_channel, a_channel, b_channel = SplitChannels(lab_image)

        // CLAHE enhances contrast in localized dark/bright background areas
        clahe = CreateCLAHE(clipLimit=3.0, tileGridSize=(8,8))
        enhanced_l = clahe.apply(l_channel)

        enhanced_lab = MergeChannels(enhanced_l, a_channel, b_channel)
        preprocessed_img = ConvertToRGB(enhanced_lab)

        // Resize image to multiples of 32 required by EAST network
        new_w = RoundToNearestMultiple(orig_w, 32)
        new_h = RoundToNearestMultiple(orig_h, 32)
        r_w = orig_w / Float(new_w)
        r_h = orig_h / Float(new_h)

        resized_img = ResizeImage(preprocessed_img, width=new_w, height=new_h)

        // Step 2: Model Loading and Inference
        net = LoadEASTTextDetectionModel(east_model_path)
        blob = CreateImageBlob(resized_img, swapRB=TRUE, crop=FALSE)
        net.setInput(blob)

        // Output layers: Scores map (text probability) and Geometry map (boxes)
        (scores, geometry) = net.forward(["feature_fusion/Conv_7/Sigmoid",
        "feature_fusion/concat_3"])
```

```

        // Step 3: Decode Geometry & Extract Text Region Proposals
        (rectangles, confidences) = DecodeEASTOutputs(scores, geometry,
min_confidence_score)

        // Apply Non-Maximum Suppression to refine overlapping text boxes
        final_boxes = ApplyNonMaximumSuppressionForRotatedBoxes(rectangles,
confidences, nms_threshold=0.4)

        // Step 4: Visualization Logic
        canvas = CopyImage(image)
        FOR EACH box IN final_boxes DO:
            // Rescale coordinates back to original image dimensions
            x1 = Round(box.x1 * r_w)
            y1 = Round(box.y1 * r_h)
            x2 = Round(box.x2 * r_w)
            y2 = Round(box.y2 * r_h)
            angle = box.angle

            // Draw rotated polygon bounding area for scene text
            polygon_pts = GetRotatedBoxVertices(x1, y1, x2, y2, angle)
            DrawPolygon(canvas, polygon_pts, color=BLACK, thickness=2)
            DrawFilledTransparentPolygon(canvas, polygon_pts, color=BLACK,
alpha=0.2)
        END FOR

        SaveImage("detected_scene_text.jpg", canvas)
        DisplayImage("Scene Text Detection", canvas)
    END FUNCTION
END SYSTEM

```